

String Manipulation Operations

Q. What is a String in Data Structures? Explain the common string manipulation operations with examples.

📌 **Definition to Remember**

In C, a **String** is a one-dimensional array of characters terminated by a special **null character** (`'\0'`). String manipulation refers to the set of operations used to modify, analyze, or compare strings. The `<string.h>` library in C provides built-in functions for all major string operations.

1. String Representation in Memory

```
char str[] = "HELLO";
Memory:  H | E | L | L | O | \0
Index:   0  1  2  3  4  5
```

- The **null character** `'\0'` marks the end of the string.
- String size in memory = length + 1 (for `'\0'`).

2. Common String Manipulation Operations

Function	Operation	Description
<code>strlen(s)</code>	Length	Returns number of characters (excluding <code>'\0'</code>)
<code>strcpy(dest, src)</code>	Copy	Copies <code>src</code> string into <code>dest</code>
<code>strcat(dest, src)</code>	Concatenation	Appends <code>src</code> to the end of <code>dest</code>
<code>strcmp(s1, s2)</code>	Comparison	Returns 0 if equal; negative if <code>s1 < s2</code> ; positive if <code>s1 > s2</code>
<code>strrev(s)</code>	Reverse	Reverses the string in place
<code>strupr(s)</code>	Uppercase	Converts all characters to uppercase

3. Code Example

```
#include <stdio.h>
#include <string.h>

int main() {
    char str1[20] = "Hello";
    char str2[20] = "World";
    char dest[40];

    // 1. Length
    printf("Length: %lu\n", strlen(str1));    // Output: 5

    // 2. Copy
    strcpy(dest, str1);
    printf("Copy: %s\n", dest);              // Output: Hello

    // 3. Concatenation
    strcat(dest, " ");
    strcat(dest, str2);
    printf("Concat: %s\n", dest);            // Output: Hello World

    // 4. Comparison
    if (strcmp(str1, str2) == 0)
        printf("Equal\n");
    else
        printf("Not Equal\n");              // Output: Not Equal

    return 0;
}
```

★ Must-Write Points (for 10 marks)

1. A string in C is a character array terminated by **null character** `'\0'`.
2. String manipulation uses the `<string.h>` library in C.
3. `strlen(s)` → counts characters excluding `'\0'`.
4. `strcpy(dest, src)` → copies one string to another.
5. `strcat(dest, src)` → appends src to the end of dest.
6. `strcmp(s1, s2)` → returns 0 if equal; lexicographic comparison.
7. Strings are used in text processing, data validation, and pattern matching (KMP, Rabin-Karp).

⚡ Quick Recall

String → char array + `'\0'` → `strlen` (length) → `strcpy` (copy) → `strcat` (join) → `strcmp` (compare, 0=equal)
→ `strrev` (reverse) → `<string.h>`



Custom String Manipulation Algorithms & C Codes

1. Custom String Operations: `strlen`, `strcpy`, `strcat`, `strrev` (C Code)

```
#include <stdio.h>

// Custom strlen
int my_strlen(const char str[]) {
    int len = 0;
    while (str[len] != '\0') len++;
    return len;
}

// Custom strcpy
void my_strcpy(char dest[], const char src[]) {
    int i = 0;
    while (src[i] != '\0') {
        dest[i] = src[i];
        i++;
    }
    dest[i] = '\0';
}

// Custom strcat
void my_strcat(char dest[], const char src[]) {
    int i = my_strlen(dest);
    int j = 0;
    while (src[j] != '\0') {
        dest[i++] = src[j++];
    }
    dest[i] = '\0';
}

// Custom String Reverse
void my_strrev(char str[]) {
    int n = my_strlen(str);
    for (int i = 0; i < n / 2; i++) {
        char temp = str[i];
        str[i] = str[n - i - 1];
        str[n - i - 1] = temp;
    }
}

int main() {
    char s1[50] = "Data";
    char s2[] = " Science";

    printf("Length of s1: %d\n", my_strlen(s1));

    my_strcat(s1, s2);
    printf("Concatenated String: %s\n", s1);

    my_strrev(s1);
    printf("Reversed String: %s\n", s1);
}
```

```
    return 0;
}
```

2. Naive Pattern Matching Algorithm (C Code)

```
#include <stdio.h>
#include <string.h>

void naivePatternSearch(char text[], char pattern[]) {
    int n = strlen(text);
    int m = strlen(pattern);

    for (int i = 0; i <= n - m; i++) {
        int j;
        for (j = 0; j < m; j++) {
            if (text[i + j] != pattern[j]) break;
        }
        if (j == m) {
            printf("Pattern found at index %d\n", i);
        }
    }
}

int main() {
    char text[] = "AABAACAADAABAAABAA";
    char pattern[] = "AABA";

    printf("Text: %s\nPattern: %s\n\n", text, pattern);
    naivePatternSearch(text, pattern);

    return 0;
}
```